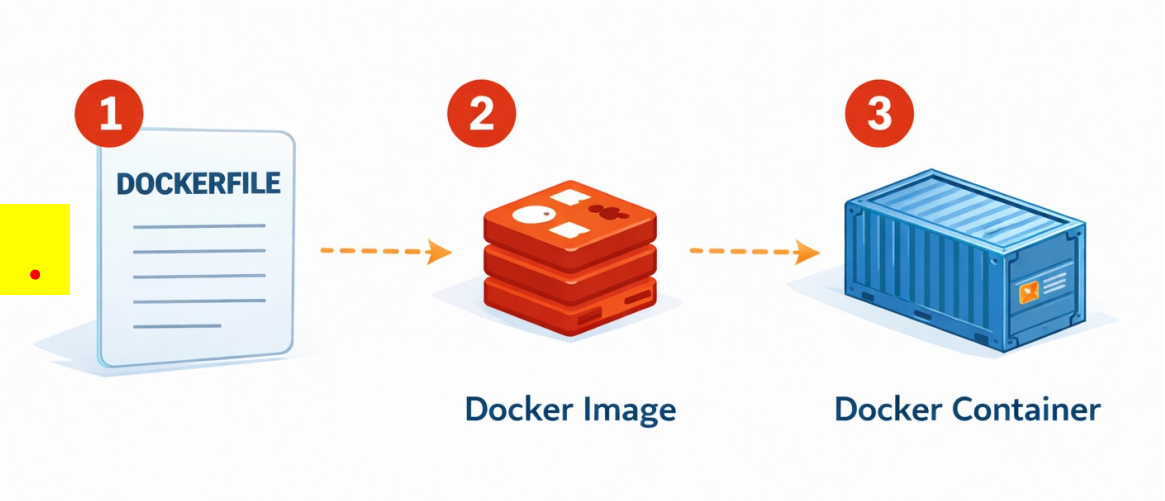
GIỚI THIỆU

Là một tệp tin văn bản chứa tập hợp các chỉ dẫn để tự động xây dựng một Docker Image

Quy trình 3 bước:

- Write: Soạn thảo các lệnh trong tệp Dockerfile.
- Build: Chạy `docker build -t [tên_image] .` để tạo Image.
- Run: Dùng `docker run` để biến Image thành Container hoạt động.



CÁC TỪ KHOÁ CƠ BẢN THƯỜNG DÙNG

Lệnh	Chức năng chính	Ví dụ
FROM	Chọn "móng nhà" (Base Image)	`FROM openjdk:17`
WORKDIR	Thiết lập thư mục làm việc (như lệnh cd)	`WORKDIR /app`
COPY/ADD	Chép code từ máy vào trong Image	`COPY . .`
RUN	Thực thi lệnh khi đang build (cài đặt thư viện)	`RUN mvn package`
ENV	Thiết lập biến môi trường	`ENV DB_URL=localhost`
EXPOSE	Khai báo cổng ứng dụng sẽ chạy	`EXPOSE 8080`
CMD/ENTRYPOINT	Lệnh mặc định khi khởi động Container	`CMD ["java", "-jar", "app.jar"]`

DEMO

VÍ DỤ MINH HOẠ

```
# --- GIAI ĐOẠN 1: BUILD ---
FROM maven:3.9.11-eclipse-temurin-25 AS builder
WORKDIR /app
COPY . .
# Biên dịch code thành file .jar (bỏ qua chạy thử test để build nhanh)
RUN mvn package -DskipTests
```

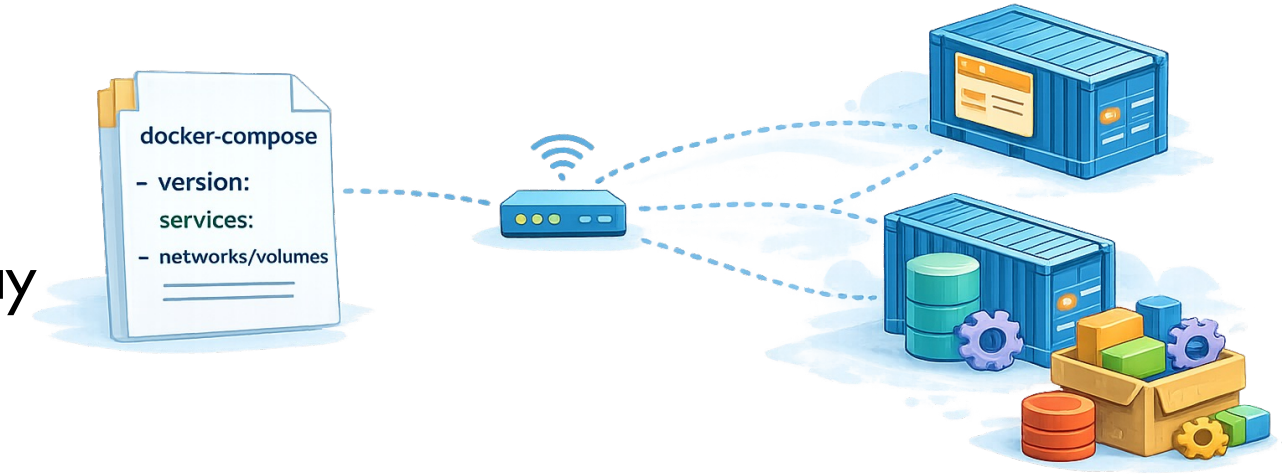
```
# --- GIAI ĐOẠN 2: RUN ---
FROM eclipse-temurin:25-jre-jammy
WORKDIR /app
# Chỉ lấy file .jar từ builder (GĐ1)
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
# Lệnh "bật công tắc" để chạy ứng dụng
ENTRYPOINT ["java", "-jar", "app.jar"]
```

GIỚI THIỆU

Là công cụ giúp định nghĩa và quản lý ứng dụng Docker đa container (multi-container).

Cấu trúc tệp docker-compose.yml:

- version: Phiên bản định dạng của Docker Compose.
- services: Danh sách các container cần chạy (ví dụ: web, db).
- networks/volumes: Cấu hình mạng chung và lưu trữ dữ liệu bền vững.



CÁC TỪ KHOÁ CƠ BẢN THƯỜNG DÙNG

Từ khóa	Ý nghĩa	Ví dụ thực tế
`version`	Xác định phiên bản cấu hình của Docker Compose (thường dùng 3.8 hoặc mới hơn).	`version: '3.8'`
`services`	Bắt đầu danh sách các container (dịch vụ) bạn muốn chạy.	`services:`
`build`	Chỉ định đường dẫn tới thư mục chứa Dockerfile để tự động build image.	`build: .`
`image`	Tên image để sử dụng (tải từ Docker Hub hoặc đặt tên cho image vừa build).	`image: buiphukhuyen/tool:8.0`
`ports`	Ánh xạ cổng (Port Forwarding) từ máy thật vào container.	`- "8080:8080"`

Từ khóa	Ý nghĩa	Ví dụ thực tế
`environment`	Thiết lập các **biến môi trường** (cấu hình DB, mật khẩu, Token...).	`MYSQL_ROOT_PASSWORD: 123`
`volumes`	Đồng bộ thư mục máy thật và container để **lưu trữ dữ liệu vĩnh viễn**.	`- ./data:/var/lib/mysql`
`depends_on`	Thiết lập **thứ tự khởi chạy** (App phải đợi Database chạy xong mới lên).	`depends_on: [db]`
`networks`	Tạo mạng ảo để các container giao tiếp nội bộ với nhau.	`networks: [backend]`
`restart`	Chính sách tự động khởi động lại nếu container bị lỗi/sập.	`restart: always`

`docker-compose up`	Build, tạo và khởi chạy toàn bộ các dịch vụ trong file .yml.
`docker-compose stop/down`	Dừng/Xoá các container đang chạy

DEMO

VÍ DỤ MINH HOẠ

```yaml
services:
  app-java:                # Tên dịch vụ tự đặt
    build: .               # Tìm Dockerfile ở thư mục hiện tại để build
    ports:
      - "8080:8080"        # Mở cổng cho sinh viên truy cập localhost:8080
    depends_on:
      - db-mysql           # Chờ DB sẵn sàng rồi mới chạy App
    environment:
      # Các biến chuẩn Spring Boot để tạo DataSource
      SPRING_DATASOURCE_URL: jdbc:mysql://db-mysql:3306/demo_docker_db?useSSL=false&allowPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root

  db-mysql:
    image: mysql:8.0       # Tải thẳng image từ Docker Hub
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: demo_docker_db
    volumes:
      - db_data:/var/lib/mysql # Dữ liệu DB không bị mất khi xóa container

volumes:
  db_data:                 # Khai báo volume dùng chung
```